

HW06 – Test Summary Report

Sinh viên: Hà Bảo Ngọc — 23127300, nhóm N08 **Môn học:** CS423 / CSC15003 – Kiểm thử Phần mềm
Ngày: 2026-08-23 **Nhánh:** HW06/23127300

Cấu trúc tài liệu theo mẫu Test Summary Report chuẩn ISTQB (softwaretestinghelp.com), điền bằng số liệu thật của HW06 — không phải template còn bỏ trống. Đây là báo cáo **bổ sung**, không thay thế `test-cases/test-summary.xlsx` (bảng số liệu HW06 §14 yêu cầu) hay mục Test Summary trong `README.md` §2.

1. Mục đích tài liệu

Tài liệu tổng hợp toàn bộ hoạt động kiểm thử API đã thực hiện cho HW06 trên hệ thống EShop: phạm vi, số liệu thực thi, loại kiểm thử đã áp dụng, môi trường/công cụ, bài học rút ra, khuyến nghị, thực hành tốt, tiêu chí hoàn thành và kết luận — dành cho người đọc (giảng viên/TA) cần nắm toàn cảnh mà không phải đọc từng báo cáo con.

2. Tổng quan hệ thống được kiểm thử (SUT)

EShop — ứng dụng thương mại điện tử tiếng Việt dùng cho mục đích luyện tập kiểm thử (<https://github.com/ttbhanh/eshop-sut>). Backend Node.js/Express + SQLite, expose REST API theo `api-specification.md`. Ba API được phân công cho sinh viên này, mỗi API thuộc một pool khác nhau:

Pool	FR	Endpoint	Chức năng
A	FR-01	POST <code>/api/register</code>	Đăng ký tài khoản
B	FR-08	POST <code>/api/checkout</code> (+ GET <code>/api/orders/:id</code> , PUT <code>/api/admin/orders/:id/status</code>)	Thanh toán, đơn hàng, state machine FR-10
C	FR-14	GET/POST/PUT/DELETE <code>/api/categories</code>	Quản lý danh mục (CRUD, admin)

3. Phạm vi kiểm thử

Trong phạm vi: Kiểm thử API (black-box) cho ba endpoint trên — domain partition mọi tham số, chuyển trạng thái (FR-10 với FR-08), bảo mật SEC-01–SEC-07, và schema đúng với đặc tả phản hồi.

Ngoài phạm vi: Giao diện web/mobile (FR-02–07, FR-09, FR-11–13, FR-15–19, Pool D); kiểm thử hiệu năng/tải; kiểm thử các API không thuộc ba endpoint được phân công.

Không kiểm thử được: Không có mục nào bị chặn do phụ thuộc bên ngoài — SUT chạy local, không phụ thuộc dịch vụ thứ ba.

4. Số liệu (Metrics)

4.1 Test case thiết kế vs thực thi

API	AI sinh	Sinh viên bổ sung	Tổng thiết kế	Newman assertions	Đạt	Không đạt
FR-01 Register	40	5	45	45	45	0
FR-08 Checkout	35	5	40	76	76	0
FR-14 Category	40	7	47	112	112	0
Tổng	115	17	132	233	233	0

Số assertion (233) cao hơn số case thiết kế (132) vì: (a) một số case data-driven kiểm tra nhiều hơn 1 assertion (status + schema), và (b) các request setup/lifecycle trong folder state/security (đăng ký, đăng nhập, seed dữ liệu) cũng chạy qua Newman và được đếm là assertion nhưng không phải case TC-* độc lập. Chi tiết từng API: `reports/main-report.md §3`.

4.2 Audit AI-generated test case (VALID / INVALID / INCOMPLETE)

API	VALID	INCOMPLETE	INVALID	Tổng AI-generated
FR-01 Register	28	9	3	40
FR-08 Checkout	25	3	7	35
FR-14 Category	21	13	6	40
Tổng	74	25	16	115

Chi tiết lý do từng case: `test-cases/FR-{01,08,14}-*/audit.md`.

4.3 Defect Summary — theo Severity

Severity	Số lượng	Bug ID
Critical	3	BUG-FR01-003, BUG-FR08-001, BUG-FR14-001
Major	2	BUG-FR01-001, BUG-FR01-002
High	2	BUG-FR08-002, BUG-FR08-003
Medium	2	BUG-FR14-002, BUG-FR14-004
Low	1	BUG-FR14-003
Tổng	10	

4.4 Defect Distribution — theo Module

Module	Số bug
FR-01 Register	3
FR-08 Checkout	3
FR-14 Category	4

Toàn bộ 10 bug đã được báo cáo tại `bug-reports/BUG-*.md` và mở GitHub Issue kèm ảnh chụp (#252–#261) — chi tiết: `README.md §2.4`.

5. Loại kiểm thử đã thực hiện

- **Domain / Boundary Value Testing** — mọi tham số của cả 3 endpoint (email, password, name, total_amount, shipping_address, category name...), theo lớp tương đương và giá trị biên (rỗng, thiếu field, quá dài, unicode, số âm/0/thập phân).
- **State-Transition Testing** — FR-10 order lifecycle (pending → confirmed → shipping → delivered , các transition bất hợp lệ, terminal-state violation) trong folder FR-08 State & Security .
- **Security Testing (SEC-01–SEC-07)** — SQL injection, XSS payload, thiếu/sai token, leo thang quyền (role escalation), IDOR, mass assignment — cho cả 3 API.
- **Schema / Contract Testing** — đối chiếu chính xác cấu trúc response (success và error path) với `api-specification.md` .
- **Regression (qua CI)** — toàn bộ suite chạy lại tự động mỗi lần push (`.github/workflows/hw06-newman.yml`), đảm bảo thay đổi collection/data không phá vỡ case đã pass.
- **Data-Driven Testing** — Postman Collection Runner với data file (`.json / .csv`) cho các folder theo tham số; folder state/lifecycle chạy một lần với case cố định.

6. Môi trường & công cụ kiểm thử

Hạng mục	Chi tiết
SUT	eshop-sut (Node.js/Express + SQLite), chạy local tại <code>http://localhost:3000 / http://127.0.0.1:3000</code>
Test tool	Postman collection v2.1 (<code>api/collections/eshop-hw06.postman_collection.json</code>)
Test runner	Newman CLI + <code>newman-reporter-htmlextra</code> (HTML report), headless
CI/CD	GitHub Actions, runner <code>ubuntu-latest</code> , Node 20 — boot SUT + chạy Newman mỗi push
Anti-cheat	Header <code>X-Student-Id: 23127300</code> gán tự động qua pre-request script cấp collection cho mọi request
Bug tracking	GitHub Issues (repo <code>lmchkhi/CS423-CSC15003-Testing-N08</code>)

7. Bài học rút ra (Lessons Learned)

1. **AI lạc quan theo đặc tả, không theo hành vi SUT thật.** Khi sinh case, AI mặc định các ràng buộc "một REST API chuẩn nên có" (unique email, password complexity, 404 cho resource không tồn tại, RBAC cho thao tác admin) mà SUT này không hề implement. Bước audit

(VALID/INVALID/INCOMPLETE) là nơi duy nhất bắt được sai lệch này trước khi nó trở thành case "pass" giả — 16/115 case AI-generated (14%) bị đánh INVALID vì lý do này (§4.2).

2. **AI có thể bịa cả bằng chứng quy trình, không chỉ bịa kết quả.** Phiên bản đầu của `prompt-log.md` được AI viết lại từ trí nhớ thay vì trích verbatim từ transcript gốc — chỉ phát hiện được nhờ đối chiếu với ký ức thực tế của sinh viên, không phải qua kiểm tra tự động. Đã dựng lại bằng script trích xuất trực tiếp từ JSONL transcript (`reports/tools/extract-prompt-log.py`). Xem `reports/ai-audit-report.md` Entry #10.
3. **Số liệu tự đối chiếu tay dễ lệch giữa nhiều tài liệu, kể cả sau khi audit xong.** Trong một đợt rà soát chéo, phát hiện bảng "Tóm tắt audit" ở cuối mỗi `audit.md` không khớp với chính bảng chi tiết ngay phía trên nó trong cùng file — và `reports/main-report.md` lại có một con số thứ ba, khác cả hai. Ba nguồn cho cùng một số liệu FR-01 audit (28/9/3 thật vs 24/9/7 vs 27/4/9) không nguồn nào khớp nguồn nào cho tới khi được đếm lại thủ công từ bảng gốc. Bài học: "đã được con người audit" không tự động nghĩa là mọi số liệu tổng hợp phải sinh từ đó cũng đúng — vẫn cần đối chiếu số cộng dồn với dữ liệu gốc, không chỉ đối chiếu từng case.

8. Khuyến nghị (Recommendations)

- Tính tổng số liệu (case count, audit tally, severity distribution) bằng script chạy trên dữ liệu gốc (bảng case, `audit.md` , Newman JSON report) thay vì gõ tay vào nhiều tài liệu độc lập, để tránh lặp lại tình huống ở mục 7.3.
- Với SUT: bổ sung ràng buộc `UNIQUE` cho `users.email` và `categories.name` , middleware kiểm tra `role` cho endpoint `admin-only` , kiểm tra `affectedRows / result.changes` sau `UPDATE / DELETE` , và validate `total_amount / state-transition` ở tầng backend — chi tiết từng khuyến nghị theo bug: `bug-reports/BUG-*.md` § Recommended Fix.
- Bổ sung test rate-limiting (chưa có trong phạm vi 3 API này) nếu mở rộng kiểm thử FR-01 sang chống abuse/DoS.

9. Thực hành tốt đã áp dụng (Best Practices)

- **Agent Skill tái sử dụng** — pipeline sinh test 7 bước (`parse_spec` → `partition_param` → `security_cases` → `schema_cases` → `state_cases` → `emit_markdown` → `emit_data_json`) được đóng gói thành `.claude/skills/api-test-generator/` , có thể áp dụng lại cho các FR khác ngoài phạm vi HW06 này (§7 report thiết kế: `reports/test-generator-design.md`).
- **CI theo folder, tách data-driven khỏi state/lifecycle** — tránh lỗi giả (request lifecycle bị lặp theo iteration khi chạy chung folder `-d`), giữ suite xanh ổn định thay vì phải rerun thủ công (`reports/ci-cd-report.md`).
- **Hai run CI mẫu (pass + fail có chủ đích)** — chứng minh pipeline thực sự bắt lỗi, không chỉ là "chạy cho có" (`reports/ci-cd-report.md` §3).
- **Bug report black-box thuần** — chỉ dùng bằng chứng request/response quan sát được, không tham chiếu mã nguồn SUT, giữ tính khách quan của kiểm thử hộp đen.

10. Tiêu chí hoàn thành (Exit Criteria)

Tiêu chí	Đạt?
Toàn bộ test case đã thiết kế được thực thi	✅ 213 request / 233 assertion, 0 fail
Mọi defect Critical/Major/High đã được ghi nhận và báo cáo	✅ 7/7 (3 Critical, 2 Major, 2 High) — <code>bug-reports/</code> , GitHub Issues #252-#261
Mọi defect Medium/Low đã được ghi nhận	✅ 3/3 (2 Medium, 1 Low)
Có bằng chứng thực thi thật (không fake/localhost giả)	✅ Host <code>127.0.0.1:3000</code> , header <code>X-Student-Id</code> xác nhận qua console log
CI/CD chạy lại được suite, có run pass và run fail mẫu	✅ <code>reports/ci-cd-report.md</code>

Không có defect nào bị chặn do thiếu môi trường hoặc phụ thuộc ngoài; không có case nào phải hoãn.

11. Kết luận

Đây là bài tập kiểm thử (không phải quyết định release sản phẩm), nên "Go-Live" không áp dụng theo nghĩa gốc — kết luận tương đương là: pipeline kiểm thử đã hoàn chỉnh và chạy xanh (0/233 fail), toàn bộ 10 defect thật phát hiện được đã báo cáo đầy đủ với bằng chứng, không có defect nào bị bỏ sót đã biết mà chưa ghi nhận. Nhóm bug ưu tiên cao nhất cần SUT team xử lý trước: BUG-FR01-003 (lộ plaintext password), BUG-FR08-001 (IDOR đơn hàng), BUG-FR14-001 (broken access control danh mục) — cả ba đều Critical.

12. Thuật ngữ / Từ viết tắt

Từ viết tắt	Ý nghĩa
SUT	System Under Test — hệ thống được kiểm thử (EShop)
FR	Functional Requirement — yêu cầu chức năng, đánh số theo <code>api-specification.md</code>
SEC-01...07	Security requirement — yêu cầu bảo mật trong đặc tả API
TC	Test Case
IDOR	Insecure Direct Object Reference — lỗi truy cập trực tiếp tài nguyên người khác
RBAC	Role-Based Access Control — kiểm soát truy cập theo vai trò
CI/CD	Continuous Integration / Continuous Delivery
P0-P3	Mức ưu tiên xử lý bug, P0 cao nhất